

ナレッジベース構築スクリプト(RAG)解説ドキュメント

1. このファイルが何をしているか

指定されたプロジェクトフォルダ内のすべてのテキストファイル(.txt)を自動的に読み込み、AIが参照できる「カンニングペーパー(知識データ)」として構築するためのスクリプトです。

```
def build_knowledge_base(project_dir):  
    """フォルダ内の全.txt ファイルを自動読み込み「カンニングペーパー化」"""
```

2. 仕組み(3 ステップ)

① データベース作成

まず、kb_db = {} という空の辞書(データベースの土台)を作成します。
データ構造のイメージ:

```
{  
    "選手名鑑": {"content": "..."},  
    "規則集": {"content": "..."}  
}
```

② テキストスキャン

フォルダ内を探索し、対象となる .txt ファイルのみを自動的にスキャンして中身を読み込み、データベースへ登録していきます。

```
for filename in os.listdir(project_dir):  
    if not filename.endswith(".txt"): continue # .txt のみ対象  
  
    with open(filepath, "r", encoding="utf-8") as f:  
        content = f.read() # ファイル全体を読み込む
```

```
kb_db[file_key] = {"content": content}    # 辞書に登録する
```

③ システムプロンプト構築

読み込んだすべてのファイル内容を結合し、AIが「知識データ」として直接認識できるシステムプロンプトの形式に整形して返却します。

```
rag_context_parts.append(content_with_path)    # ファイル内容を結合  
full_content = "\n—————".join(rag_context_parts)    # すべてを結合する  
return {"system": system_prompt + full_content}    # AI に渡す形で返却
```

3. 使い方の例

ローカルの実行環境でターゲットとなるディレクトリに移動し、スクリプトを実行するだけで自動的にデータベースが構築されます。

```
cd local-ai-project  
python rag_local.py # コマンド実行でスキャン & DB 構築完了！
```

出力結果の例：

```
✓ RAG データベース構築完了  
登録ファイル数: 5
```

4. プロ技の活用法

- ① コンテキストの提示法【/file rag_local.py】
このファイルをAIチャットに提示した上で、"build_db() 関数が何をしているか、素人にもわかるように解説" のようにピンポイントで指示を出すと効果的です。
- ② Insert コマンドによる事故防止 (Ctrl + Command + V)
万が一エラーが出た場合は、エラー文をAIに投げて解決策を出してもらい、生成された修正コードを Insert 機能で直接上書きします。これにより手動コピペによる貼り付けミスや事故をゼロにできます。
- ③ 新しい話題への切り替え【New Thread (+ アイコン)】
解説や質問の方向性をガラッと変えたいときは、New Thread (+ アイコン) をクリックします。

過去の文脈やキャッシュのノイズを完全にリセットして、最速のレスポンス初速でやり取りをスタートできます。